

AI Critique

During this assignment, AI was useful for generating a large number of API test cases and identifying common testing categories such as valid inputs, invalid inputs, boundary values, authentication, injection attacks, malformed requests, and response validation.

However, the generated test cases were not always accurate or sufficiently aligned with the actual System Under Test (SUT). For example, AI sometimes proposed overly broad expected status codes such as 200/400, 400/404, or 401/403, instead of determining the exact expected behavior from the API implementation and specification. Some generated cases were also redundant or tested scenarios that were not explicitly required by the API contract.

In addition, AI focused heavily on common security and input-validation patterns but could miss application-specific behavior, such as valid order-state transitions, ownership of an order, or the actual behavior of authentication middleware. This happened because AI primarily reasoned from the API description and general testing knowledge rather than observing the complete runtime behavior of the SUT. It also cannot reliably determine how a particular backend implementation handles ambiguous or unsupported inputs without executing the tests.

Human review was therefore necessary to validate the generated cases, correct unrealistic expectations, and add missing scenarios. The main principle I learned is that AI should be treated as a test-design assistant rather than an authoritative source of truth. AI can improve coverage and accelerate test-case generation, but the tester remains responsible for verifying assumptions against the specification, source code, and actual execution results. Effective collaboration with AI requires a continuous cycle of **AI generation** → **human review** → **test execution** → **failure analysis** → **refinement**.